

SHAYVERI CORE - Pre-Phase 1 Summary

Compute Orchestration & Rendering Engine · Java / Spring Boot backend

Everything done and everything to understand before Module 1 is written. July 2026.

What This Document Is

A complete recap of the project up to the start of Phase 1: what was scaffolded, why it is shaped the way it is, what every folder is for, what git ignores and why, and the current git state. The build plan (the seven modules, milestones, acceptance criteria) is deliberately NOT in this document. It lives independently and solely in `Idea_Generation/plan.txt`; `README.md` is a short front page pointing to it.

Project location:

```
/Users/averi/Undergrad/CS/Java/Java projects/  
SHAYVERI CORE (Compute Orchestration & Rendering Engine)
```

1. What Was Built (Phase 0 Only)

Phase 0 is deliberately just the skeleton: the app boots, API-key security works, and one ping endpoint exists. No telemetry, jobs, nodes, or config logic has been written yet. That is Phases 1 through 4.

Files created

File	Purpose
<code>build.gradle.kts</code> / <code>settings.gradle.kts</code>	Gradle Kotlin DSL build. Spring Boot 3.5.0, Java 21 toolchain, starters: web, websocket, data-mongodb, data-redis, validation, actuator, security.
<code>gradlew</code> , <code>gradlew.bat</code> , <code>gradle/wrapper/</code>	Committed Gradle wrapper pinned to 8.14.2 so anyone can build without installing Gradle.
<code>docker-compose.yml</code>	Local dev infra: mongo:7 and redis:7-alpine, one command (<code>docker compose up -d</code>).
<code>src/main/resources/application.yml</code>	Mongo/Redis URIs, virtual threads enabled, three API keys from env vars with dev defaults, Open Cloud key/universe placeholders, actuator exposure.
<code>CoreApplication.java</code>	Entry point. Three lines: <code>main()</code> calls <code>SpringApplication.run()</code> .
<code>common/PingController.java</code>	<code>GET /api/ping</code> returns <code>{"service":"shayveri-core","status":"ok"}</code> .
<code>common/ApiKeyAuthFilter.java</code>	<code>OncePerRequestFilter</code> reading the <code>X-API-Key</code> header and mapping it to a role.
<code>common/ApiKeyProperties.java</code> , <code>ApiKeyRole.java</code>	Loads the three keys from config; <code>ROBLOX</code> / <code>NODE</code> / <code>DASH</code> role enum.
<code>config/SecurityConfig.java</code>	Stateless, CSRF off, <code>/actuator/health</code> public, everything else requires a valid key.
<code>README.md</code>	Short front page: description, quick start, status. The plan itself lives in <code>Idea_Generation/plan.txt</code> .
<code>.gitignore</code>	Excludes <code>.gradle/</code> , <code>build/</code> , <code>.idea/</code> , <code>*.iml</code> and other generated files. Section 5 covers every entry in depth.

2. Decisions and Deviations From the Plan

Spring Boot 3.5.0 instead of the pinned 3.3.4

The plan pinned Boot 3.3.4, but that version has dropped out of Spring Initializr's supported range (it now requires 3.5.0 or newer). Boot 3.5.x satisfies the same 'Boot 3.3+' requirement.

Hand-built scaffold instead of the start.spring.io curl command

The hosted Spring Initializr service was returning HTTP 500 errors server-side, even for minimal requests, so the project was scaffolded manually with identical results (same dependencies, layout, and config).

Gradle installed via Homebrew (9.6.1)

Installed only to generate the wrapper; the project itself uses the committed wrapper (8.14.2), so the global install is not a build dependency.

Storage behind interfaces (added after the asdb discussion)

Each module will define a small store interface for its persistence needs (JobStore, HeartbeatStore, ConfigStore); Mongo/Redis implementations are adapters behind it. Business logic never touches a Mongo/Redis API directly, which keeps each storage slot independently swappable, e.g. for asdb (custom Rust database) as a v2 milestone.

3. Security Model

3.0 What Spring Security is

Spring Security is a library - one of the dependencies in build.gradle.kts (spring-boot-starter-security). It is the security framework for Spring applications: a large, battle-tested toolkit of prebuilt authentication and authorization machinery that you configure instead of writing yourself.

The problem it solves

Every web backend needs the same security plumbing: intercept every request, figure out who is calling, decide whether they are allowed, reject cleanly if not, and defend against a catalog of classic attacks (session hijacking, CSRF, header injection, timing attacks on credential comparison). Writing that yourself is thousands of lines where any mistake is a vulnerability. Spring Security is that plumbing written once, audited for roughly twenty years, and used by banks and governments.

How it works mechanically: the filter chain

Before any request reaches a controller, Spring Security passes it through an ordered stack of filters. Each filter does one job: one might check a login session, one might validate a token, one blocks the request entirely if nobody authenticated it. Think of it as airport security between the door (the HTTP port) and the gates (the controllers) - a sequence of checkpoints, each specialized.

Where it shows up in this project, concretely

First: the moment the dependency was added, it locked everything down by default - that is why /api/ping demands a key even though PingController contains zero security code. Deny-by-default is the framework's philosophy: things are opened explicitly rather than each endpoint remembering to protect itself.

Second: SecurityConfig.java is us configuring the framework. Each line is an instruction: sessionCreationPolicy(STATELESS) = do not create login sessions; csrf.disable() = skip browser-form protection (meaningless for an API keyed by header); requestMatchers("/actuator/health").permitAll() = this one is public; anyRequest().authenticated() = everything else requires an identity.

Third: ApiKeyAuthFilter is our one custom checkpoint inserted into the chain. Spring Security ships filters for passwords, OAuth, and more, but 'API key in a header' is our own scheme, so we wrote one filter and slotted it into the stack (addFilterBefore in the config). It reads X-API-Key and, if valid, tells the framework 'this request is ROBLOX' (or NODE / DASH). The framework handles everything after that, including rejecting requests where no filter vouched for an identity.

The division of labor: we wrote ~40 lines (which key maps to which role); the framework does everything else (enforcing it on every request, the rejection responses, the attack defenses, and the role machinery that later lets 'only NODE can claim jobs' be one line). This is also why the 'security lives in two places' rule below works: the filter chain and the config are not our invention - they are Spring Security's architecture, and the rule is a commitment to use it as designed instead of scattering ad-hoc checks through module code.

3.1 The current model (v1)

Role	Who	Env var	Dev default
ROBLOX	Roblox game servers	SHAYVERI_KEY_ROBLOX	dev-roblox-key
NODE	Lab node agents	SHAYVERI_KEY_NODE	dev-node-key
DASH	React dashboard	SHAYVERI_KEY_DASH	dev-dash-key

Keys are sent in the X-API-Key header. No sessions, no CSRF, fully stateless. /actuator/health is the only public endpoint. This is intentionally minimal: v1 serves ~9 lab machines you physically control and two human users on a private deployment. Building key management UIs now would be speculation; what matters is that growing later is cheap. The rest of this section is how that is guaranteed.

3.2 Security lives in exactly two places

Place one: the filter chain (authentication - 'who are you')

Spring Security models security as an ordered stack of filters that every request passes through before it can reach any controller. Today the stack contains one custom filter, ApiKeyAuthFilter, which reads the header and attaches a role to the request. The stack is the extension point: adding rate limiting, HMAC request signing, or IP allowlists later means inserting another filter into the stack. Existing filters, controllers, and modules are untouched - this layered chain-of-responsibility design is why the same framework scales from hobby projects to banks.

Place two: SecurityConfig (authorization - 'what may you do')

Endpoint-to-role rules are declared centrally in one file: only ROBLOX may post telemetry, only NODE may claim jobs, only DASH may edit config. Tightening or changing a rule is a one-line change in one place, never a hunt through module code.

3.3 The binding discipline rule

No module ever inspects credentials itself. No controller or service reads the X-API-Key header, compares keys, or branches on caller identity - by the time a request reaches module code, security has already

decided. If a credential check ever appears inside ingress/, jobs/, or any module package, that is an architecture violation, with the same weight as the plan's five development rules. This single discipline is what keeps every future security change confined to the two places above.

3.4 The planned seam: ApiKeyResolver

One small refactor is scheduled before Module 1 lands on top of the static setup: an ApiKeyResolver interface - resolve(rawKey) returns a principal of { role, label } - with StaticApiKeyResolver as the v1 implementation reading the three env-var keys. The filter will depend only on the interface. This is the same pattern as development rule 5 (storage behind interfaces): when keys later move to Mongo with hashing and caching, only a new resolver implementation is written; the filter, the config, and all seven modules never change. The label field also future-proofs the Module 7 audit log: today it answers 'a NODE did this', later 'node-7 did this' with zero downstream changes.

3.5 Evolution roadmap - every future need has exactly one landing spot

Future security need	Where it lands
Per-client keys (each node/place its own key) + revocation	New ApiKeyResolver implementation
Key rotation managed from the dashboard	New resolver implementation + a small DASH-only API
Rate limiting per key	New filter in the chain
HMAC request signing / TLS	New filter / deployment concern
OAuth dashboard login (already on the v2 list)	Parallel filter path for DASH only
Tighter endpoint rules	One-line SecurityConfig changes

The pattern to notice: every row lands in the resolver, the filter stack, or the config - never in module code. That is the definition of security that scales: needs will grow, and each one arrives as an addition in a known place instead of a rewrite.

4. Folder Structure

4.1 The full tree

```

SHAYVERI CORE (...)/
|-- README.md          <- short front page; GitHub renders it on the repo page
|-- build.gradle.kts    <- WHAT to build: dependencies, Java version, plugins
|-- settings.gradle.kts <- project name (one line)
|-- docker-compose.yml  <- recipe to start local MongoDB + Redis in one command
|-- gradlew / gradlew.bat <- scripts that download and run Gradle (Mac/Linux + Windows)
|-- gradle/wrapper/     <- config + jar the gradlew scripts use; commit, never edit
|-- .gitignore          <- (hidden) tells git which folders to never commit
|-- Idea_Generation/    <- storage: not part of the program
|   |-- plan.txt        <- THE PLAN, standing alone
|   |-- docs/           <- construction blueprints (module1_blueprint.txt)
|   |-- PrePhase1_Summary.pdf <- this PDF, plus future working notes
|-- src/                <- ALL the actual code lives under here
|   |-- main/java/dev/shayveri/core/ <- production code
|   |   |-- CoreApplication.java    <- entry point, main()
|   |   |-- common/                 <- shared: API-key auth, ping endpoint
|   |   |-- config/                 <- Spring configuration (security rules)
|   |   |-- ingress/                <- Module 1: telemetry intake (empty yet)
|   |   |-- nodes/                  <- Module 2: lab node registry (empty yet)

```

		-- jobs/	<- Module 3: job queue + lifecycle	(empty yet)
		-- overrides/	<- Module 4: game config overrides	(empty yet)
		-- egress/	<- Module 5: Roblox Open Cloud	(empty yet)
		-- realtime/	<- Module 6: WebSocket hub	(empty yet)
		-- observability/	<- Module 7: metrics + audit log	(empty yet)
		-- main/resources/	<- non-code files the app reads: application.yml	
		-- test/java/	<- tests, mirroring the main layout	(empty so far)
		-- .gradle/	<- (hidden, GENERATED) Gradle's cache:	never commit
		-- .idea/	<- (hidden, GENERATED) IntelliJ settings:	never commit
		-- build/	<- (GENERATED) compiled output:	never commit

4.2 Subfolders outside src/ - what each one is for in the program

gradle/

The build system's bootstrap. Its one subfolder, `wrapper/`, holds a tiny jar and a properties file that pin the exact Gradle version (8.14.2). When anyone runs `./gradlew`, the script uses these to download that exact Gradle automatically. Abstracted purpose: it is what makes the build reproducible on any machine with zero setup. Committed once, never edited by hand.

Idea_Generation/

The storage folder; not part of the program, so no abstracted program purpose to explain. At its root sits `plan.txt`, the build plan (system context, all seven module specs, milestones, acceptance criteria; the sole home of the plan) - the contract every other component (dashboard, node agents, Roblox scripts) is built against; when the plan changes, `plan.txt` itself is edited. `docs/` holds construction blueprints (`module1_blueprint.txt`), and this PDF and future working notes sit at the root beside `plan.txt`.

.gradle/, .idea/, build/ (the generated three)

Machine output, not source. `.gradle/` is Gradle's private working cache for this project; `.idea/` is IntelliJ's per-machine settings (window layout, local run configs); `build/` is where compiled `.class` files and packaged jars land when the project builds. All three are recreated automatically from scratch whenever needed, which is exactly why git ignores them (section 5). Never open, never edit, never commit.

4.3 The dividing line: src/ versus everything else

Everything outside `src/` exists to build, run, document, or support the program: the build definition (the `.kts` files), the build bootstrap (`gradlew` + `gradle/`), the local infrastructure recipe (`docker-compose.yml`), the documentation (`README.md`, `Idea_Generation/`), and machine output (the generated three). None of it ships logic.

Everything inside `src/` IS the program: the Java that compiles into the running service, plus the config files it reads at startup. If the repo were a restaurant, `src/` is the kitchen; everything else is the lease, the supplier contracts, and the instruction manuals.

4.4 Inside src/

The three fixed branches

`src/main/java` holds production code, `src/main/resources` holds non-code files the app reads at startup (`application.yml`), and `src/test/java` holds tests, mirroring the main layout. This split is the standard Gradle/Maven layout: the build tool finds code by location, not by configuration, which is why compilation worked with zero 'where is my code' setup.

Why `dev/shayveri/core` - packages are folders

Java package names map one-to-one onto folder paths, and the compiler enforces the match: a class declared 'package dev.shayveri.core.common;' must physically live in dev/shayveri/core/common/. The reverse-domain prefix (dev.shayveri, like com.google or org.springframework) guarantees these classes can never collide with a library class that happens to share a name. The middle folders (dev/, shayveri/) never contain files directly; they only spell out the namespace.

The module packages

Under dev.shayveri.core the code is organized package-by-module, not by layer: config/ and common/ hold Spring setup and shared pieces (the API-key filter, the ping endpoint), and the seven module folders (ingress, nodes, jobs, overrides, egress, realtime, observability) each own one module from the plan - its controllers, services, store interfaces, and documents together in one place. All seven are empty until their phase is built. Git does not track empty directories, so they will not appear in the repo until their first class lands.

Making IntelliJ display it sanely

IntelliJ collapses the empty middle folders: the Project panel shows one node 'dev.shayveri.core' instead of three nested folders. If it does not, click the gear icon in the Project panel and enable 'Compact Middle Packages'. Finder is the wrong viewer for a Java project; the IDE view is the real one.

4.5 The mental model - three zones

Zone 1, files you write: everything in src/, build.gradle.kts, settings.gradle.kts, docker-compose.yml, README.md, Idea_Generation/.

Zone 2, files written once and then left alone: gradlew, gradlew.bat, gradle/wrapper/, .gitignore.

Zone 3, machine-generated, never opened, never committed: .gradle/, build/, .idea/.

4.6 Why the root files cannot move into a subfolder

Gradle looks for build.gradle.kts, settings.gradle.kts, gradlew, and gradle/ at the project root, period; git reads .gitignore from the repo root; 'docker compose up' looks for docker-compose.yml in the current directory. These locations are contracts with the tools, not stylistic choices. Every Spring Boot repo on GitHub has this identical root set, and people reading the repo expect it there.

5. The .gitignore, Explained

The .gitignore is a hidden text file at the repo root listing path patterns git must pretend do not exist: they never show in git status, never get staged by git add. The governing principle: commit what humans write, ignore what machines derive. Anything a tool can regenerate from the committed sources has no business in history - it bloats every clone, changes on every build, and causes merge conflicts that mean nothing. Entry by entry:

.gradle/

Gradle's private per-project cache: dependency metadata, task state, file hashes it uses to decide what needs recompiling. Regenerated on every build, different on every machine.

build/

All build output: compiled .class files, processed resources, test reports, packaged jars. Fully derivable from `src/` by running `./gradlew build`. Committing compiled output alongside source is redundant and drifts out of sync. (This is exactly the junk the drag-and-drop GitHub upload accidentally included.)

`!gradle/wrapper/gradle-wrapper.jar`

The one deliberate exception, marked by the leading `!` which means do-NOT-ignore. Jars are normally build junk, but the wrapper jar is the bootstrap that lets `./gradlew` work on a machine with no Gradle installed, so it must be committed. The `!**/src/main/**/build/` style entries are the same idea: if a source folder legitimately contained a directory named 'build', it would not be swept up by the `build/` rule.

`.idea/`, `*.iml`, `*.iws`, `*.ipr`, `out/`

`.idea/` holds IntelliJ's per-machine project settings: window layout, local run configurations, per-user editor state. The `*.iml/*.iws/*.ipr` patterns are IntelliJ's older module/workspace/project file formats, and `out/` is IntelliJ's own compile output folder (its equivalent of `build/` when you use the IDE's builder instead of Gradle). All of it is personal to one machine and one user; committing it causes pointless conflicts between collaborators' IDE preferences.

`.apt_generated`, `.classpath`, `.factorypath`, `.project`, `.settings`, `.springBeans`, `.sts4-cache`, `bin/`

The same story for Eclipse and Spring Tool Suite users: these are Eclipse's project metadata and output folders. Nobody on this project uses Eclipse today, but the standard Spring Boot gitignore covers it so a future collaborator's IDE files never pollute the repo.

`.vscode/`

VS Code's per-machine workspace settings. Same reasoning as `.idea/`.

`HELP.md`

Spring Initializr drops a boilerplate `HELP.md` with links into generated projects; it is noise, so the standard gitignore excludes it.

`.DS_Store`

macOS Finder's hidden per-folder metadata (icon positions, view options). Appears in any folder you browse; means nothing to anyone else.

6. Why the Code Looks 'Empty'

Spring Boot inverts program flow compared to a plain `Main.java` project: you write small annotated classes (`@RestController`, `@Configuration`) and the framework finds them, wires them together, and calls them when HTTP requests arrive. The server loop, socket handling, and JSON parsing all come from the framework dependencies declared in `build.gradle.kts`. This is the standard style for every Spring Boot backend and was locked in by the plan's stack decisions. Substantial readable logic arrives with Module 1.

The deep folders (`.gradle/`, `build/`, `gradle/wrapper` contents) are build tooling and caches, not source code. They are gitignored and never edited by hand.

7. Verification Status

Verified: `./gradlew compileJava` passes cleanly.

Not yet verified: Docker is not installed on this machine, so docker compose up, ./gradlew bootRun, GET /actuator/health, and the key-gated /api/ping check have not been run against live Mongo/Redis. That is the remaining Phase 0 acceptance work.

8. Git - Current State and What Gets Committed

Where things stand right now

The local repo is initialized with identity configured, and the clean Phase 0 commit exists locally (18 files, branch main, remote wired to github.com/AveriWylie/SHAYVERI-CORE-). The GitHub side currently holds a drag-and-drop web upload instead: everything nested one folder too deep, compiled build/ junk included, hidden files (.gitignore) missing, and gradlew's executable permission lost. The fix is a one-time login then a force push, which replaces the uploaded mess with the clean commit:

```
gh auth login          # GitHub.com > HTTPS > Yes > Login with a web browser
git push --force -u origin main
```

After that, the day-to-day loop is: git add . then git commit -m "message" then git push.

What belongs in a commit

Commit: src/, gradle/wrapper/ (including the jar), gradlew, gradlew.bat, build.gradle.kts, settings.gradle.kts, docker-compose.yml, README.md, Idea_Generation/, .gitignore.

Never commit (and .gitignore enforces it): .gradle/, build/, .idea/, *.iml. If git status ever shows these as addable, something is wrong with the .gitignore.

9. External APIs the Code Consumes

The dividing principle (development rule 6): if we code it ourselves, it lives in its module and needs no catalog - the module IS the documentation of ownership. Everything else the code touches is someone else's API, and those surfaces are what must be known. Each blueprint unit carries a 'Consumes' line naming exactly what it uses; this section is the full catalog. The pattern to notice across all of them: every library hands you either annotations to mark your classes, or one entry-point object with a fluent API, and the framework does the heavy lifting behind it. Learning Spring is mostly learning which annotation or entry point maps to which job - this catalog is the complete vocabulary the project needs through Phase 4.

9.1 Spring Security - the HttpSecurity configuration DSL

The API behind SecurityConfig.java: an object Spring hands us with chainable methods. What exists versus what we call:

API call	What it does	Used?
http.csrf(c -> c.disable())	turn off browser-form protection	yes
http.sessionManagement(... STATELESS)	never create server sessions	yes
http.authorizeHttpRequests(...)	open the URL-rules block	yes
.requestMatchers(path).permitAll()	make paths public	yes (health)
.requestMatchers(path).hasRole(R)	restrict path to a role	from Module 1 on

API call	What it does	Used?
<code>.anyRequest().authenticated()</code>	everything else needs identity	yes
<code>http.addFilterBefore(f, cls)</code>	insert our checkpoint into the chain	yes
<code>http.build()</code>	produce the finished chain	yes
<code>formLogin()</code> / <code>oauth2Login()</code> / <code>httpBasic()</code> / <code>cors()</code>	login styles we do not use	no (OAuth maybe v2)

The filter-writing API (behind ApiKeyAuthFilter)

OncePerRequestFilter - extend it, override `doFilterInternal(request, response, chain)`.

`request.getHeader("X-API-Key")` - plain Servlet API.

`SecurityContextHolder.getContext().setAuthentication(...)` - the 'I vouch for this request' call.

`UsernamePasswordAuthenticationToken` + `SimpleGrantedAuthority` - the objects carrying identity and role.

`chain.doFilter(...)` - pass to the next checkpoint.

9.2 The rest of the consumed surfaces, library by library

Library	Entry points / annotations	Used for
Spring core (DI)	@Configuration, @Bean, @Service, @Component; constructor injection; @ConfigurationProperties	Spring constructs and wires our classes; binds application.yml blocks to objects (ApiKeyProperties)
Spring Web	@RestController, @GetMapping/@PostMapping, @RequestBody, @PathVariable, @RequestParam, ResponseEntity, @RestControllerAdvice + @ExceptionHandler	every controller; global error handling (blueprint C2)
Jakarta Validation	@NotNull, @Min, @Positive on DTO fields; @Valid on the controller parameter	request validation (A1/A2); failures become 400s via C2
Spring Data MongoDB	@Document, @Id, @Indexed(expireAfter="7d"); interface X extends MongoRepository - Spring GENERATES the implementation: save(), findById(), findAll(), derived queries from method names; MongoTemplate as escape hatch	documents + TTL (A3/A4); store adapters (A6)
Spring Data Redis	StringRedisTemplate: opsForValue().set(key, val, Duration.ofSeconds(45)); opsForList() operations	heartbeats with auto-expiry (Module 2); job queues + atomic claim (Module 3)
WebSocket / STOMP	@EnableWebSocketMessageBroker; WebSocketMessageBrokerConfigurer (registerStompEndpoints, configureMessageBroker); SimpMessagingTemplate.convertAndSend(topic, payload)	the realtime hub (B1-B3); convertAndSend is the one send method RealtimePublisher wraps
Jackson (JSON)	deliberately invisible - records convert to/from JSON automatically; @JsonProperty only when a field name must differ from the JSON key	every request/response body, every broadcast payload
RestClient	restClient.post().uri(url).header(...).body(obj).retrieve()	outbound Open Cloud calls (Module 5)
Micrometer	MeterRegistry: registry.counter("shayveri.ingest").increment(), gauge(), Timer	custom metrics (Module 7)
Java virtual threads	spring.threads.virtual.enabled=true (already on); Executors.newVirtualThreadPerTaskExecutor()	cheap async persistence/broadcast off the request thread (A7)

Library	Entry points / annotations	Used for
Test harnesses	@SpringBootTest, MockMvc/WebTestClient; spring-security-test; Testcontainers (@Testcontainers, @Container, MongoDBContainer); STOMP test client	the whole test plan (section D of the blueprint)

10. Next Step: Phase 1

Module 1 (Ingress - telemetry intake) plus a minimal Module 6 (Realtime WebSocket hub). Definition of done: a fake Luau script (curl loop) posts telemetry and it is visible on a raw WebSocket client. Full endpoint specs are in the plan: Idea_Generation/plan.txt. Construction blueprint: Idea_Generation/docs/module1_blueprint.txt.